

Exercícios autorais

Análise Léxica e Sintática

Conteúdo autoral derivado do lesson model aprovado; a bibliografia é complementar.

1. Análise do código fonte

Aplique “Análise do código fonte” a um pequeno trecho de entrada ou gramática criado por você. Explique o resultado passo a passo e indique como verificaria se ele está correto.

Considere a entrada $x = 10$; Primeiro, o analisador recebe o código-fonte. Em seguida, identifica seus elementos: x como identificador, $=$ como operador de atribuição, 10 como número e $;$ como delimitador. Depois, esses elementos podem ser utilizados pela análise sintática para verificar se estão em uma sequência válida. Para verificar se o resultado está correto, deve-se comparar os elementos reconhecidos com as regras léxicas e sintáticas da linguagem e verificar se a entrada é aceita sem erros.

2. FNB na descrição da sintaxe

Analise o erro conceitual: alguém tratou “FNB na descrição da sintaxe” como uma etapa isolada, sem relação com a entrada ou com as demais etapas. Explique o problema e reescreva a conclusão.

O erro está em tratar a FNB como uma etapa isolada. A FNB está relacionada à descrição da sintaxe porque representa uma forma de especificar as regras que determinam quais sequências de símbolos são válidas. Portanto, ela deve ser considerada juntamente com a entrada e com o processo de análise sintática.

Conclusão reescrita: A FNB é utilizada para descrever formalmente a sintaxe de uma linguagem, permitindo que uma entrada seja comparada às regras gramaticais e que se determine se ela possui uma estrutura sintaticamente válida.

3. Analisador léxico e analisador sintático

Construa um exemplo mínimo que demonstre “Analisador léxico e analisador sintático”. Identifique a entrada, a decisão realizada e a saída esperada.

Considere a entrada $soma = 5$;

O analisador léxico identifica $soma$ como identificador, $=$ como operador, 5 como número e $;$ como delimitador. O analisador sintático recebe esses tokens e verifica se a sequência segue uma estrutura válida, por exemplo, uma atribuição. A saída esperada é a aceitação da entrada, caso a sequência esteja de acordo com a gramática.

4. Separação da análise léxica e da análise sintática

Compare o comportamento de um analisador com e sem considerar “Separação da análise léxica e da análise sintática”. Aponte o impacto no reconhecimento ou no diagnóstico.

Com a separação, o analisador léxico primeiro transforma os caracteres da entrada em tokens e o analisador sintático trabalha sobre esses tokens. Isso facilita a identificação dos erros: um caractere inválido pode ser identificado na análise léxica, enquanto uma sequência de tokens em ordem incorreta pode ser identificada na análise sintática.

Sem essa separação, as duas responsabilidades ficam misturadas, tornando o reconhecimento e o diagnóstico dos erros mais complexos.

5. Análise Léxica

Explique “Análise Léxica” para um colega usando uma definição, um exemplo autoral e uma relação com outro conceito da aula.

A análise léxica é responsável por examinar os caracteres da entrada e agrupá-los em unidades léxicas que serão utilizadas como tokens. Por exemplo, na entrada idade = 20, o analisador pode reconhecer idade como identificador, = como operador e 20 como número. Ela se relaciona diretamente com o conceito de lexema e token, pois os lexemas encontrados na entrada são classificados em categorias de tokens para serem utilizados posteriormente pela análise sintática.

6. Lexema e token

Aplice “Lexema e token” a um pequeno trecho de entrada ou gramática criado por você. Explique o resultado passo a passo e indique como verificaria se ele está correto.

Considere a entrada: total = 25

O analisador encontra primeiro o lexema total e o classifica como um token de identificador. Depois encontra = e o classifica como operador de atribuição. Por fim, encontra 25, que é um lexema classificado como número. Para verificar o resultado, deve-se conferir se cada sequência de caracteres foi corretamente identificada e associada à categoria de token correspondente.

7. Abordagens para construir o analisador léxico

Analise o erro conceitual: alguém tratou “Abordagens para construir o analisador léxico” como uma etapa isolada, sem relação com a entrada ou com as demais etapas. Explique o problema e reescreva a conclusão.

O erro está em considerar as abordagens de construção do analisador léxico como algo independente da entrada e das outras etapas. A abordagem escolhida precisa produzir os tokens necessários para que a análise sintática possa trabalhar corretamente.

Conclusão reescrita: As abordagens para construir o analisador léxico são formas de implementar o reconhecimento dos elementos da entrada, devendo produzir uma sequência de tokens adequada para a etapa de análise sintática.

8. Classes de caracteres

Construa um exemplo mínimo que demonstre “Classes de caracteres”. Identifique a entrada, a decisão realizada e a saída esperada.

Considere a entrada abc123.

O analisador pode utilizar classes de caracteres para diferenciar letras e dígitos. Ao encontrar a, b e c, identifica caracteres pertencentes à classe de letras; ao encontrar 1, 2 e 3, identifica a classe de dígitos. Como a sequência começa com letras e continua com dígitos, ela pode ser reconhecida como um identificador, dependendo das regras da linguagem.

Entrada: abc123

Decisão: classificar os caracteres e determinar o tipo da sequência.

Saída: token de identificador.

9. Reconhecimento de palavras reservadas

Compare o comportamento de um analisador com e sem considerar “Reconhecimento de palavras reservadas”. Aponte o impacto no reconhecimento ou no diagnóstico.

Considere a palavra while. Com o reconhecimento de palavras reservadas, o analisador verifica se while pertence à tabela de palavras reservadas e gera um token específico de palavra reservada. Sem essa verificação, poderia classificá-la simplesmente como um identificador.

Isso afeta o reconhecimento porque palavras reservadas possuem funções específicas na linguagem e não devem ser tratadas como identificadores comuns.

10. Funções getChar, addChar e lookup

Explique “Funções getChar, addChar e lookup” para um colega usando uma definição, um exemplo autoral e uma relação com outro conceito da aula.

Essas funções podem participar do funcionamento de um analisador léxico. getChar obtém o próximo caractere da entrada; addChar adiciona o caractere reconhecido ao lexema em construção; e lookup verifica se uma sequência corresponde a um símbolo ou palavra reservada conhecida.

Por exemplo, ao analisar if, getChar obtém i e depois f, addChar constrói o lexema if e lookup pode verificar que if é uma palavra reservada. Essas funções estão relacionadas diretamente à análise léxica, pois auxiliam na transformação dos caracteres da entrada em tokens.

11. Objetivos da análise sintática

Aplique “Objetivos da análise sintática” a um pequeno trecho de entrada ou gramática criado por você. Explique o resultado passo a passo e indique como verificaria se ele está correto.

Considere a entrada x = 10;. O objetivo da análise sintática é verificar se os tokens produzidos pela análise léxica estão organizados de acordo com as regras da gramática. Primeiro, identifica-se x como identificador, depois = como atribuição, 10 como número e ; como delimitador. A análise verifica se essa sequência corresponde a uma construção válida, como

uma atribuição. Para verificar o resultado, deve-se comparar a sequência com a regra gramatical correspondente.

12. Análise Sintática Descendente

Analise o erro conceitual: alguém tratou “Análise Sintática Descendente” como uma etapa isolada, sem relação com a entrada ou com as demais etapas. Explique o problema e reescreva a conclusão.

O erro está em tratar a análise sintática descendente como uma etapa independente. Ela utiliza a entrada de tokens e as regras da gramática para construir a estrutura sintática partindo do símbolo inicial em direção aos símbolos da entrada.

Conclusão reescrita: Na análise sintática descendente, o analisador começa pelo símbolo inicial da gramática e procura derivar a sequência de tokens da entrada, utilizando as regras gramaticais para verificar se ela é válida.

13. Análise Sintática Ascendente

Construa um exemplo mínimo que demonstre “Análise Sintática Ascendente”. Identifique a entrada, a decisão realizada e a saída esperada.

Considere a entrada: $3 + 4$

O analisador recebe os tokens 3, + e 4. Na análise ascendente, começa-se pelos elementos da entrada e eles são agrupados de acordo com as regras da gramática até chegar ao símbolo inicial. Assim, 3 e 4 podem ser reconhecidos como expressões e, juntamente com +, formar uma expressão maior.

Entrada: $3 + 4$

Decisão: reduzir os elementos conforme as regras da gramática.

Saída: reconhecimento da entrada como uma expressão válida.

14. Deslocamento e redução

Compare o comportamento de um analisador com e sem considerar “Deslocamento e redução”. Aponte o impacto no reconhecimento ou no diagnóstico.

No analisador ascendente, o deslocamento coloca o próximo token da entrada na pilha, enquanto a redução substitui uma sequência de símbolos da pilha pelo símbolo correspondente do lado esquerdo de uma regra gramatical.

Por exemplo, em $3 + 4$, o analisador pode deslocar 3, depois deslocar + e 4, e realizar reduções conforme as regras da gramática até formar uma expressão. Sem essas operações, o analisador não teria o mecanismo necessário para construir a estrutura sintática a partir da entrada.

15. Complexidade da análise sintática

Explique “Complexidade da análise sintática” para um colega usando uma definição, um exemplo autoral

e uma relação com outro conceito da aula.

A complexidade da análise sintática está relacionada ao esforço necessário para determinar se uma sequência de tokens pertence à linguagem definida pela gramática. Por exemplo, uma gramática que exige muitas possibilidades de análise pode tornar o reconhecimento mais complexo. Isso se relaciona às diferentes técnicas de análise sintática, pois determinadas estratégias são utilizadas justamente para tornar o processo de reconhecimento mais eficiente e controlado.

16. Análise Sintática Descendente Recursiva

Aplique “Análise Sintática Descendente Recursiva” a um pequeno trecho de entrada ou gramática criado por você. Explique o resultado passo a passo e indique como verificaria se ele está correto.

Considere uma gramática simples: $E \rightarrow \text{num} + \text{num}$
e a entrada: $5 + 3$

Na análise descendente recursiva, uma função associada a E começa verificando a primeira parte da regra. Ela reconhece 5 como num, verifica o símbolo + e depois reconhece 3 como outro num. Ao final, toda a entrada foi consumida de acordo com a regra $E \rightarrow \text{num} + \text{num}$.

Para verificar se está correto, deve-se conferir se cada token foi consumido na ordem esperada e se a sequência corresponde à produção utilizada.

17. FNBE para expressões simples

Analise o erro conceitual: alguém tratou “FNBE para expressões simples” como uma etapa isolada, sem relação com a entrada ou com as demais etapas. Explique o problema e reescreva a conclusão.

O erro está em tratar a FNBE como uma etapa isolada. Ela deve estar relacionada às regras da gramática e às decisões realizadas pelo analisador para reconhecer expressões.

Conclusão reescrita: A FNBE pode ser utilizada para representar uma forma adequada de uma gramática para expressões, permitindo que o analisador descendente determine de maneira organizada qual produção deve ser utilizada para reconhecer a entrada.

18. Processamento do lado direito de uma regra

Construa um exemplo mínimo que demonstre “Processamento do lado direito de uma regra”. Identifique a entrada, a decisão realizada e a saída esperada.

Considere a regra: $E \rightarrow \text{num} + \text{num}$
e a entrada $7 + 2$.

O analisador inicia o processamento do lado direito da regra, verificando os elementos na ordem: primeiro num, depois + e finalmente outro num. A entrada $7 + 2$ satisfaz todos esses elementos.

Entrada: $7 + 2$

Decisão: verificar cada símbolo do lado direito da produção.

Saída: regra reconhecida e expressão aceita.

19. Convenção de nextToken

Compare o comportamento de um analisador com e sem considerar “Convenção de nextToken”. Aponte o impacto no reconhecimento ou no diagnóstico.

A convenção de nextToken permite indicar de forma organizada qual é o próximo token disponível para ser analisado. Com essa convenção, o analisador consegue avançar pela entrada de maneira previsível. Sem ela, o controle do próximo elemento poderia ficar menos organizado, dificultando o reconhecimento e o diagnóstico de erros.

Por exemplo, ao analisar $x + 5$, o analisador pode trabalhar sucessivamente com ID, + e NUM por meio da atualização de nextToken.

20. Lookahead

Explique “Lookahead” para um colega usando uma definição, um exemplo autoral e uma relação com outro conceito da aula.

Lookahead é a observação de um ou mais tokens seguintes da entrada antes de tomar uma decisão de análise. Por exemplo, se o analisador encontra um identificador e precisa decidir qual produção utilizar, pode observar o próximo token para determinar o caminho adequado.

Ele está relacionado à análise sintática descendente, pois o analisador pode utilizar o próximo token para escolher entre diferentes alternativas da gramática.

21. Recursão à esquerda

Aplique “Recursão à esquerda” a um pequeno trecho de entrada ou gramática criado por você. Explique o resultado passo a passo e indique como verificaria se ele está correto.

Considere a regra: $E \rightarrow E + \text{num} \mid \text{num}$

A regra apresenta recursão à esquerda porque E aparece novamente como o primeiro símbolo do lado direito de uma produção. Em uma análise descendente recursiva, isso pode fazer a função de E chamar a si mesma continuamente antes de consumir a entrada.

Para verificar o problema, basta observar que, ao tentar reconhecer E, a primeira alternativa exige novamente E antes de consumir num. Portanto, a gramática deve ser transformada para eliminar essa recursão quando utilizada por esse tipo de analisador.

22. FIRST

Analise o erro conceitual: alguém tratou “FIRST” como uma etapa isolada, sem relação com a entrada ou com as demais etapas. Explique o problema e reescreva a conclusão.

O erro está em tratar FIRST como uma etapa independente. FIRST é utilizado para auxiliar decisões de análise, especialmente quando existem alternativas de produção.

Por exemplo: $E \rightarrow \text{num} + \text{num} \mid \text{id} + \text{num}$

Os conjuntos FIRST das alternativas são diferentes:

$\text{FIRST}(\text{num} + \text{num}) = \{\text{num}\}$

$\text{FIRST}(\text{id} + \text{num}) = \{\text{id}\}$

Assim, ao observar o próximo token, o analisador consegue decidir qual produção utilizar.

Conclusão reescrita: FIRST representa os símbolos que podem aparecer inicialmente nas derivações de uma produção e é utilizado pelo analisador para escolher adequadamente entre alternativas da gramática.

23. Teste de Disjunção par a par

Construa um exemplo mínimo que demonstre “Teste de Disjunção par a par”. Identifique a entrada, a decisão realizada e a saída esperada.

Considere: $E \rightarrow \text{num} + \text{num} \mid \text{id} + \text{num}$

As duas alternativas possuem conjuntos FIRST diferentes: a primeira começa com num e a segunda com id. O teste de disjunção par a par verifica se as alternativas podem ser distinguidas pelo próximo símbolo da entrada.

Entrada: 5 + 3

Decisão: o primeiro token é num, portanto seleciona-se a primeira alternativa.

Saída: $E \rightarrow \text{num} + \text{num}$.

Para a entrada x + 3, o primeiro token seria id, selecionando a segunda alternativa.